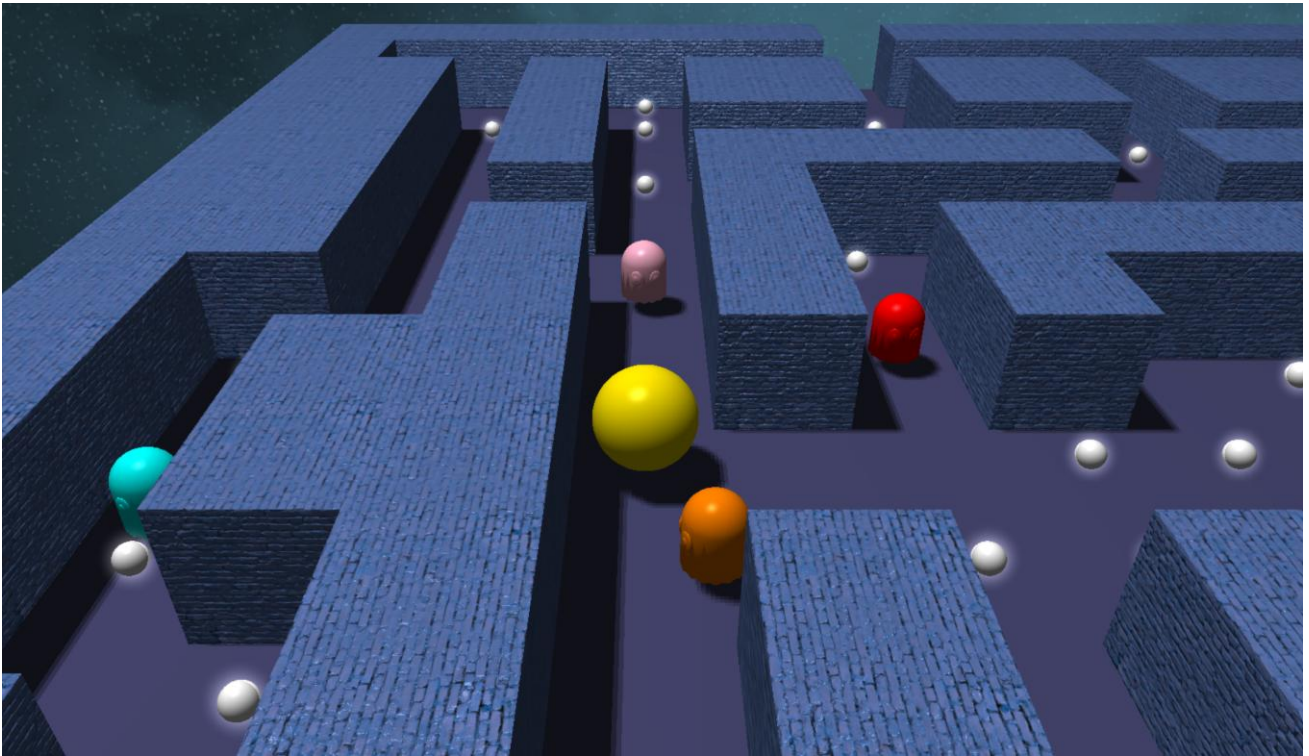


UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

PACMAN 3D:

Joc interactiv în OpenGL



Proiect opțional la disciplina: „Sisteme de Prelucrare Grafică”

Realizat de: Solomon Anastasia, grupa 1311B

Iași, 2026

Cuprins

Introducere	2
Metodologie	2
Rezultate	8
Concluzii	14
Referințe Bibliografice	15

Introducere

Proiectul constă în implementarea unui joc inspirat din clasicul Pacman, transpus într-un mediu tridimensional randat în timp real cu OpenGL. Jocul păstrează mecanicele originale: un labirint, bănuți de colectat și fantome de evitat, dar adaugă o perspectivă 3D cu camera urmăritoare, efecte grafice și o serie de tehnici de randare implementate în C++.

Motivația principală a fost să aplic cunoștințele acumulate la laboratoarele SPG într-un proiect concret și interactiv, explorând în același timp tehnici noi dincolo de cerințele minime, cu scopul de a obține un rezultat vizual coerent și o experiență de joc funcțională.

Obiective propuse:

- Implementarea unui joc Pacman funcțional cu logica completă: coliziuni cu pereți, colectare bănuți, detecție coliziune cu fantome, tunel de teleportare și stări de victorie / înfrângere.
- Randare 3D în timp real cu modele tridimensionale pentru toate personajele / obiectele prezente în joc (Pacman + bănuți: sferă, fantome: modele OBJ încărcate din fișier, labirint: cub).
- Shadow mapping cu proiecție ortografică și filtrare PCF pentru umbre fine la rezoluția 2048x2048.
- Normal mapping pe pereții labirintului, cu o lampă suplimentară pentru evidențierea detaliilor.
- Efect de strălucire (halo) pe bănuți prin billboard.
- Skybox cu textura cubemap pentru fundalul scenei.
- Optimizarea geometriei pereților prin greedy meshing, reducând semnificativ numărul de apeluri de randare.
- Mișcare fluidă bazată pe deltaTime și interpolare liniară (lerp) pentru Pacman, cameră și fantome.

Metodologie

Proiectul este scris în C++ și folosește OpenGL 4.0 prin GLEW și FreeGLUT pentru gestionarea ferestrei și a inputului. Matematica 3D este realizată cu biblioteca GLM, iar încărcarea texturilor cu stb_image. Modelele de fantome sunt încărcate dintr-un fișier OBJ printr-un loader prezentat la laborator. Shaderurile sunt organizate în perechi vertex/fragment separate: light.vert / light.frag pentru iluminare, depth.vert / depth.frag pentru shadow map, halo.vert / halo.frag pentru glow și skybox.vert / skybox.frag pentru fundal.

1. Pipeline de randare în două faze

Fiecare cadru parcurge două faze distincte. În prima fază, scena este randată din perspectiva sursei de lumină folosind shaderul de adâncime, rezultatul fiind stocat într-un framebuffer offscreen (depth map) la rezoluția 2048x2048. În a doua fază, scena este randată din perspectiva camerei jucătorului, folosind harta de adâncime generată anterior pentru calculul umbrelor.

```
// Faza 1: randare in depth map (din perspectiva luminii)
glUseProgram(depth_shader_programme);
glUniformMatrix4fv(depthUniforms.lightSpaceLoc, 1, GL_FALSE,
glm::value_ptr(lightSpaceMatrix));

glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
glClear(GL_DEPTH_BUFFER_BIT);

renderScene(depthUniforms);
glBindFramebuffer(GL_FRAMEBUFFER, 0);

// Faza 2: randare finala cu shadow map
glUseProgram(shader_programme);
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, depthMap);
renderScene(lightUniforms);
```

2. Shadow Mapping

Lumina principală este poziționată deasupra labirintului și folosește o proiecție ortografică pentru a acoperi întreaga scenă. În shaderul de fragment, adâncimea fragmentului curent este comparată cu valoarea stocată în harta de adâncime. Pentru a elimina artefactele de tip shadow acne, se aplică un bias adaptiv în funcție de unghiul dintre normala suprafeței și direcția luminii. Percentage Closer Filtering (PCF) eșantionează adâncimea în 9 puncte dintr-o grilă 3x3 și face media rezultatelor, producând o tranziție graduală la marginea umbrelor.

```
// light.frag: calcul shadow cu PCF
float ShadowCalculation(vec4 fragPosLightSpace, vec3 normal, vec3 lightDir) {
    vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
    projCoords = projCoords * 0.5 + 0.5;
    float bias = max(0.05 * (1.0 - dot(normal, lightDir)), 0.005);
    float shadow = 0.0;
    vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
    for (int x = -1; x <= 1; ++x)
```

```

        for (int y = -1; y <= 1; ++y) {
            float pcfDepth = texture(shadowMap, projCoords.xy + vec2(x, y) *
texelSize).r;
            shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
        }
        return shadow / 9.0;
    }
}

```

3. Normal Mapping

Pereții labirintului sunt texturați cu o textură și o hartă de normale. Harta de normale stochează în fiecare pixel o direcție de normală perturbată, codificată RGB, care permite simularea de detalii fine, fără geometrie suplimentară. Calculele se fac în spațiul tangențial: fiecare vertex conține poziție, normală geometrică și vector tangent, din care vertex shaderul construiește matricea TBN (Tangent-Bitangent-Normal) pentru transformarea corectă a direcțiilor de iluminare.

4. Modelul de iluminare

Iluminarea folosește modelul Phong cu două surse active simultan. Lumina principală (direcțională, poziționată deasupra labirintului) calculează componentele ambient, difuz și specular și proiectează umbre prin shadow mapping. O lampă suplimentară, orientată în direcția camerei, este folosită exclusiv pe suprafețele texturate pentru a scoate în evidență detaliile normal map-ului pe pereții vizibili direct de către jucător. Lampa nu proiectează umbre.

```

// light.frag: combinare surse de lumina
vec3 ambientComponent = vec3(0.25) * baseColor;

// Lumina principala (cu shadow)
float dotLN = max(dot(N, lightDir), 0.0);
vec3 diffuseMain = vec3(0.8) * dotLN * baseColor;
vec3 R1 = reflect(-lightDir, N);
vec3 specularMain = vec3(0.3) * pow(max(dot(R1, viewDir), 0.0), 32.0);

// Lampa de cap (fara shadow, doar pe suprafete texturate)
float headDot = max(dot(N, viewDir), 0.0);
vec3 diffuseHeadlamp = vec3(0.1) * headDot * baseColor;
vec3 specularHeadlamp = vec3(0.25) * pow(max(dot(reflect(
viewDir, N), viewDir), 0.0), 32.0);

float shadow = ShadowCalculation(FragPosLightSpace, worldNormal, worldLightDir);
vec3 finalColor = ambientComponent + (1.0 - shadow) * (diffuseMain +
specularMain) + diffuseHeadlamp + specularHeadlamp;

```

5. Efectul de strălucire (halo) a bănuților

Bănuții beneficiază de un efect de strălucire implementat fără bloom. Se desenează un dreptunghi care se rotește mereu cu fața spre cameră, peste fiecare bănuț activ (billboarding). În vertex shader (halo.vert), orientarea billboard-ului se obține extrăgând vectorii dreapta și sus direct din matricea de vizualizare. În fragment shader (halo.frag), transparența scade progresiv cu distanța față de centrul pătratului, creând un halo circular cu marginile estompate. Blending-ul aditiv (GL_ONE) face ca luminile să se adune, producând un efect de emisie vizibilă.

```
// halo.vert: billboard mereu spre camera
vec3 right = vec3(view[0][0], view[1][0], view[2][0]);
vec3 up = vec3(view[0][1], view[1][1], view[2][1]);
vec3 worldPos = centerPos + right * vPos.x * size + up * vPos.y * size;
gl_Position = projection * view * vec4(worldPos, 1.0);

// halo.frag: fade radial
float dist = length(uv);
if (dist > 1.0) discard;
float alpha = pow(1.0 - dist, 2.0);
fragColor = vec4(haloColor, alpha);

// main.cpp: blending aditiv inainte de desenare
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE);
```

6. Optimizarea pereților prin Greedy Meshing

O abordare naivă ar desena fiecare celulă de perete ca un cub separat, rezultând până la 300 de apeluri de randare per cadru pentru o matrice de 15x20. Algoritmul de greedy meshing parcurge labirintul și la fiecare celulă de perete nevizitată, extinde un segment cât mai mult posibil pe orizontală (sau pe verticală dacă nu este posibil pe orizontală). Geometria combinată este încărcată o singură dată în memoria GPU într-un buffer static și randată într-un singur apel `glDrawArrays`. Coordonatele de textură sunt scalate proporțional cu dimensiunea segmentului, evitând deformarea texturilor.

```
// Extindere pe orizontala
int w = 0;
while (c + w < MAZE_WIDTH && maze[r][c+w] == 1 && !visited[r][c+w])
    w++;

// Daca nu merge pe orizontala, incercam pe verticala
int d = (w == 1) ? 0 : 1;
if (w == 1)
    while (r + d < MAZE_HEIGHT && maze[r+d][c] == 1 && !visited[r+d][c])
        d++;

// Marcam celulele ca vizitate si adaugam un singur segment
for (int i = 0; i < d; i++)
    for (int j = 0; j < w; j++)
        visited[r+i][c+j] = true;

wallSegments.push_back({ (float)c, (float)r, (float)w, (float)d });

// UV scalate proportional: un perete lat de 3 repeta textura de 3 ori
float uW = seg.width; // scala UV pe orizontala
float uD = seg.depth; // scala UV pe adancime
```

7. DeltaTime și interpolarea liniara (Lerp)

Toate mișcările din joc sunt înmulțite cu `deltaTime`: timpul scurs de la cadrul anterior, măsurat în secunde, asigurând o viteză constantă indiferent de performanța hardware-ului. Pentru a evita teleportarea bruscă între poziții, se aplică interpolare liniară (`lerp`): poziția curentă se deplasează treptat către poziția țintă cu formula: $\text{curent} += (\text{ținta} - \text{curent}) * \text{viteza} * \text{deltaTime}$. Aceeași tehnică se aplică

și rotației unghiulare a lui Pacman și a fantomelor, cu o corecție pentru a evita rotații pe calea cea mai lungă (când diferența de unghi depășește π).

```
// Calcul deltaTime
float currentFrameTime = glutGet(GLUT_ELAPSED_TIME) / 1000.0f;
float deltaTime = currentFrameTime - lastFrameTime;
lastFrameTime = currentFrameTime;
if (deltaTime > 0.1f) deltaTime = 0.1f; // protectie la minimizare fereastra

// Lerp pozitional pentru Pacman
float speed = 10.0f * deltaTime;
pacX += (targetX - pacX) * speed;
pacZ += (targetZ - pacZ) * speed;

// Lerp unghiular cu corectia caii scurte
float diff = targetAngle - currentAngle;
if (diff > PI) diff -= 2.0f * PI;
if (diff < -PI) diff += 2.0f * PI;
currentAngle += diff * (10.0f * deltaTime);
```

8. Skybox cu Cubemap

Fundalul scenei este randat cu un skybox bazat pe o textură cubemap formată din 6 imagini PNG (dreapta, stânga, sus, jos, față, spate). Skybox-ul este desenat ultimul în pipeline, cu funcția de adâncime setată la `GL_LEQUAL` pentru a permite pixelilor din buffer să treacă testul. Componenta de translație este eliminată din matricea de vizualizare, astfel încât skybox-ul să pară infinit de departe și să nu se deplaseze odată cu camera.

```
// Desenam skybox-ul ultimul, cu GL_LEQUAL
glDepthFunc(GL_LEQUAL);
glUseProgram(skybox_shader_programme);

// Eliminam translatia din view matrix (pastram doar rotatia)
glUniformMatrix4fv(skyboxViewLoc, 1, GL_FALSE, glm::value_ptr(viewMatrix));

glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_CUBE_MAP, cubemapTexture);
glBindVertexArray(skyboxVao);
glDrawArrays(GL_TRIANGLES, 0, 36);

glDepthFunc(GL_LESS); // resetam pentru restul scenei

// skybox.vert: eliminam translatia
gl_Position = (projection * mat4(mat3(view))) * vec4(pos, 1.0).xyww;
// .xyww forteaza z = w => adancime normalizata (z/w) = 1.0
```

9. Camera și controlul cu mouse-ul

Camera este poziționată în spatele lui Pacman la o distanță și înălțime fixă, urmărind automat direcția de mișcare printr-un unghi care se mișcă prin lerp către unghiul lui Pacman. Jucătorul poate roti suplimentar camera ținând apăsat butonul stâng al mouse-ului și deplasând cursorul: delta de poziție X al mouse-ului este adăugat unghiului camerei cu o sensibilitate de 0.01 radiani per pixel. La prima acțiune de mișcare după o rotire cu mouse-ul, camera revine la poziția standard din spatele lui Pacman.

```

// Pozitia camerei in spatele lui Pacman
float finalCamAngle = cameraAngle + cameraMouseRotation;
float camX = pacX + sin(finalCamAngle) * distanceBehind;
float camZ = pacZ + cos(finalCamAngle) * distanceBehind;
viewMatrix = glm::lookAt(
    glm::vec3(camX, heightAbove, camZ), // pozitia camerei
    glm::vec3(pacX, 0.3f, pacZ),       // tinta (Pacman)
    glm::vec3(0, 1, 0));               // up

// Rotire cu mouse (callback motion)
float delta = (x - lastMouseX) * 0.01f; // 0.01 rad/pixel
cameraMouseRotation += delta;

```

10. Logica jocului

Labirintul este definit ca o matrice 2D de 15x20 celule (1 = perete, 0 = cale). Bănuții sunt plasate aleator la inițierea jocului cu o probabilitate de 50% pe fiecare celulă liberă. Pacman se mișcă pe grilă o celulă la rând, cu verificare de coliziune (poziția țintă este acceptată doar dacă celula destinație nu este perete). Jocul include un tunel de teleportare pe rândul 7 (dacă numărăm de la 0), colectare automată a bănuților la trecerea lui Pacman peste ele și detecție de coliziune cu fantomele (rază de 0.4 unități). Cele 4 fantome (Blinky, Pinky, Inky, Clyde) urmează rute de patrulare predefinite în stil round-robin.

```

// Verificare coliziune perete inainte de miscare
int gridX = (int)(nextX + 0.5f);
int gridZ = (int)(nextZ + 0.5f);
if (maze[gridZ][gridX] != 1) {
    targetX = nextX;
    targetZ = nextZ;
}

// Colectare pellet la pozitia curenta
if (pellets[currentGridZ][currentGridX]) {
    pellets[currentGridZ][currentGridX] = false;
    // verificam daca mai exista pelleti => victorie
}

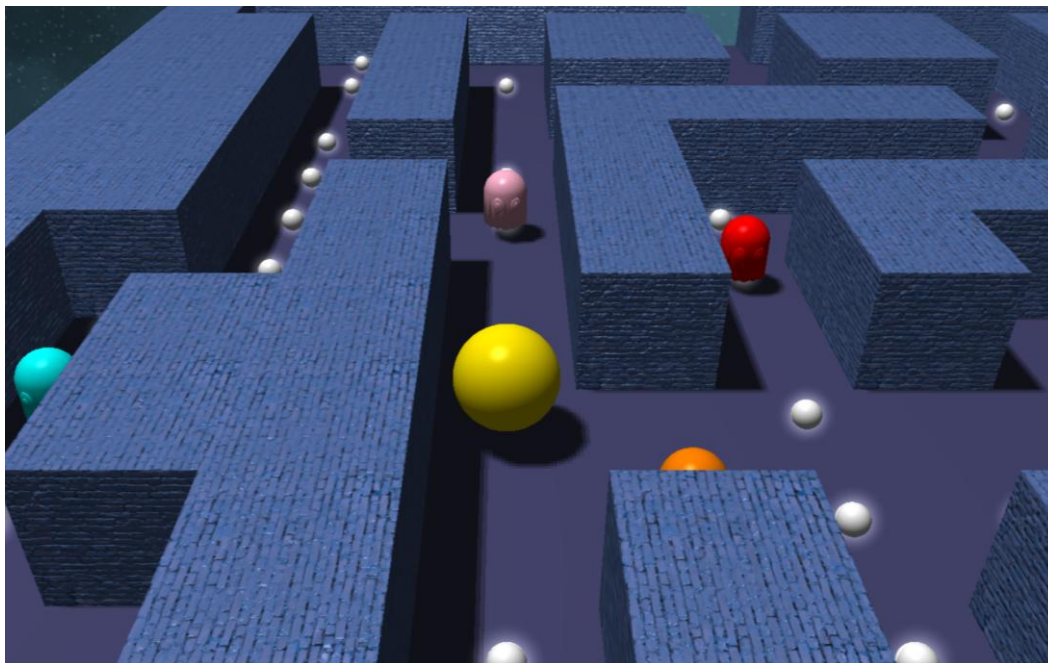
// Detectie coliziune Pacman - fantoma
float dist = glm::distance(
    glm::vec2(pacX, pacZ), glm::vec2(g.x, g.z));
if (dist < 0.4f) isGameOver = true;

// Tunel: randul 7 permite iesirea pe margini
if (gridZ == 7 && (gridX < 0 || gridX >= MAZE_WIDTH))
    targetX = nextX; // teleportare acceptata

```


Rezultate

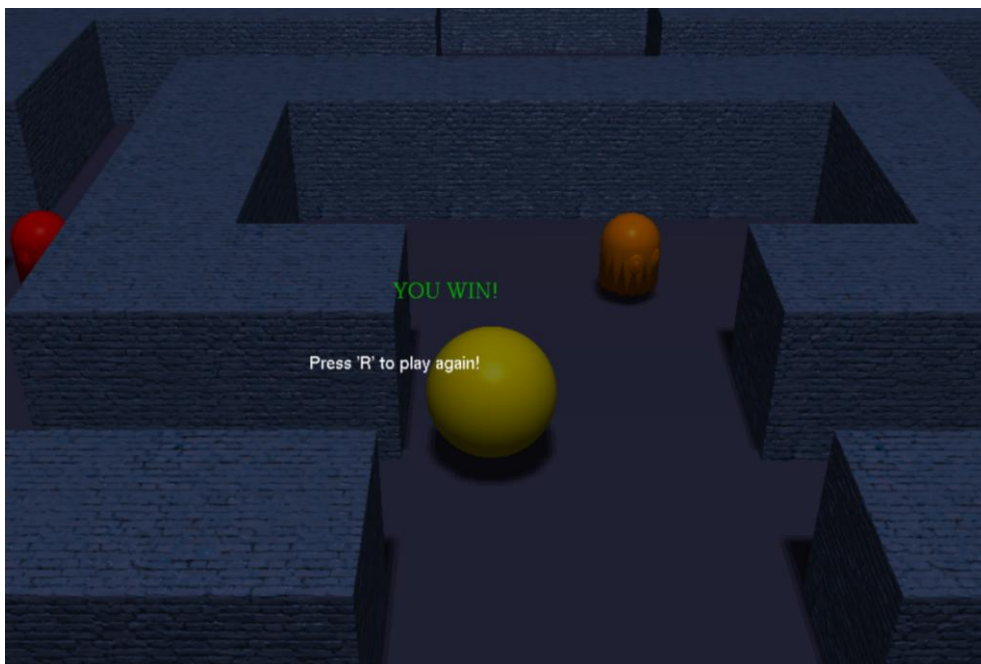
Proiectul implementează un joc Pacman 3D, cu toate funcționalitățile grafice propuse. Mai jos sunt prezentate funcționalitățile principale și capturi relevante din aplicație.



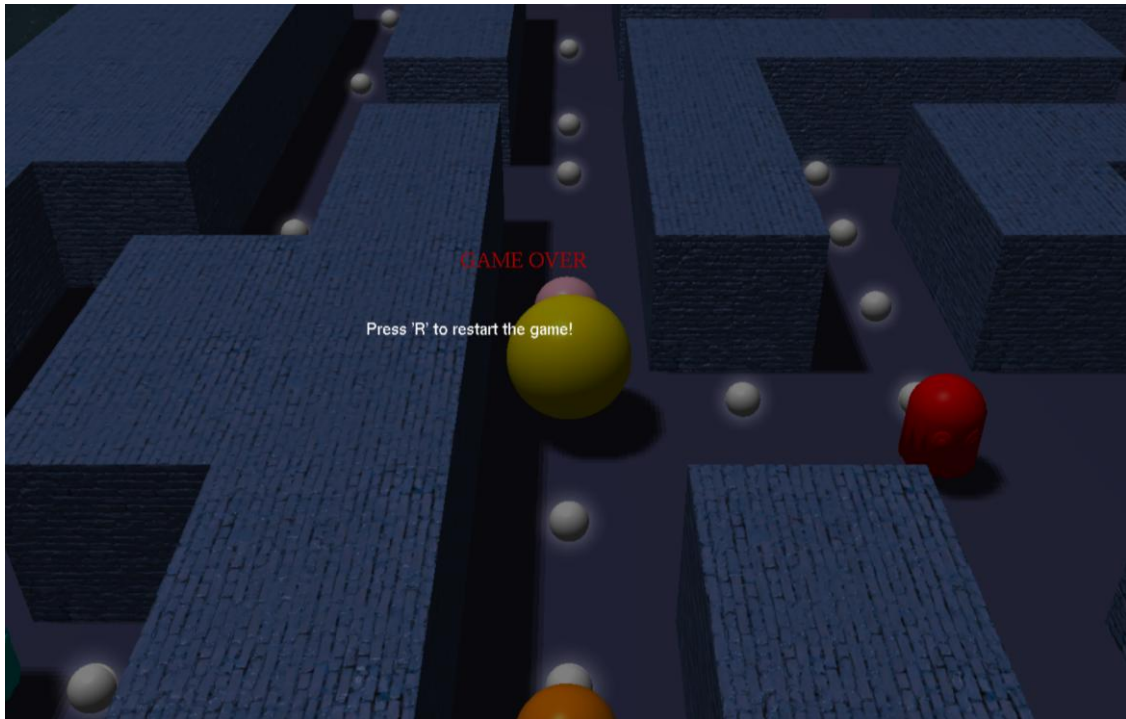
Labirintul 3D cu Pacman, bănuți și fantome

Funcționalități implementate

1. **Joc Pacman 3D funcțional:** mișcare pe grilă, colectare bănuți, detecție coliziune cu fantome, tunel de teleportare, stare de victorie și înfrângere cu mesaje pe ecran.

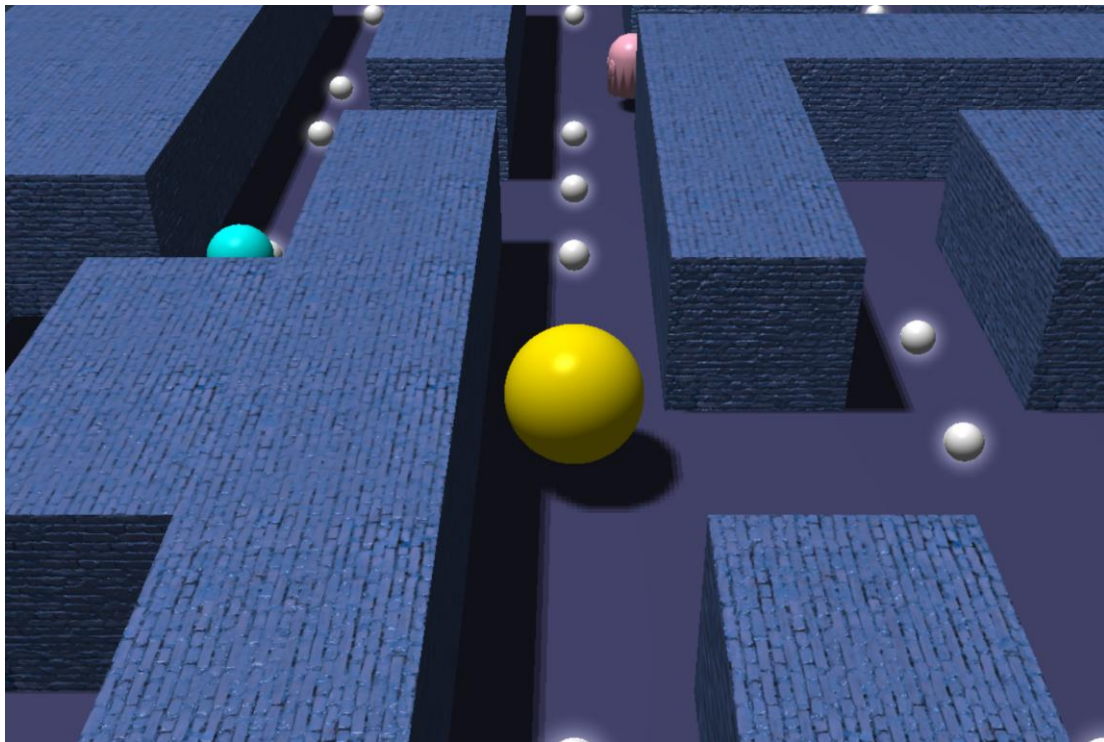


Ecran de victorie

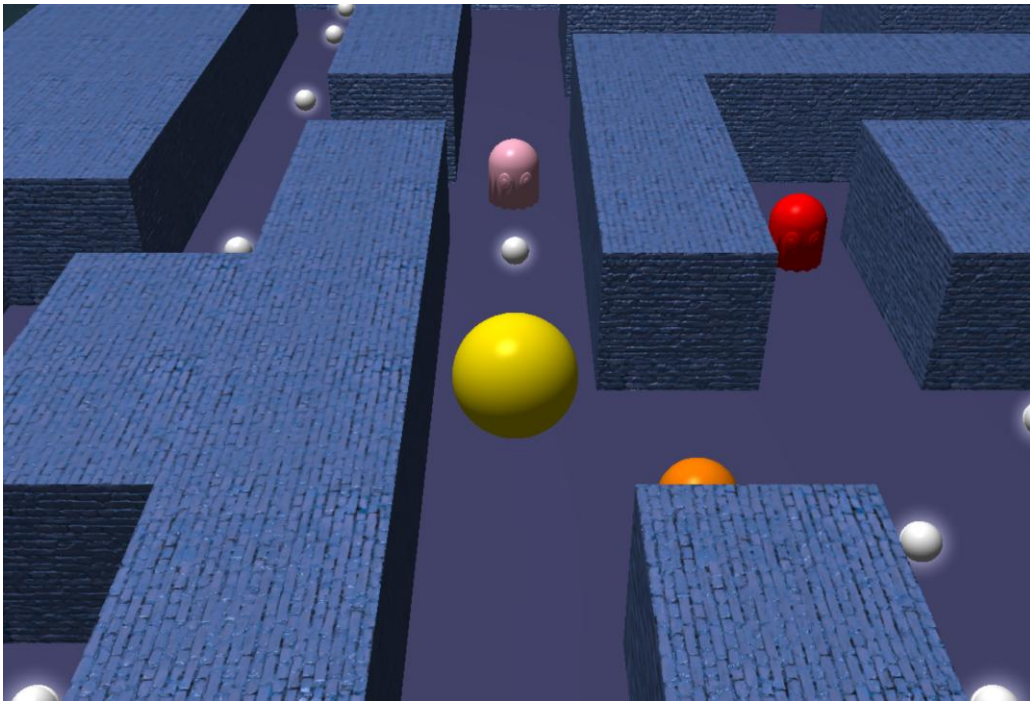


Ecran de înfrângere

2. **Personaje 3D:** Pacman reprezentat ca sferă subdivizată (nivel 3), fantome ca modele OBJ încărcate din fișier cu normală per vertex.
3. **Pereți combinați prin greedy meshing:** texturați și cu o hartă de normale.
4. **Shadow mapping cu PCF:** la rezoluția 2048x2048, cu bias adaptiv.

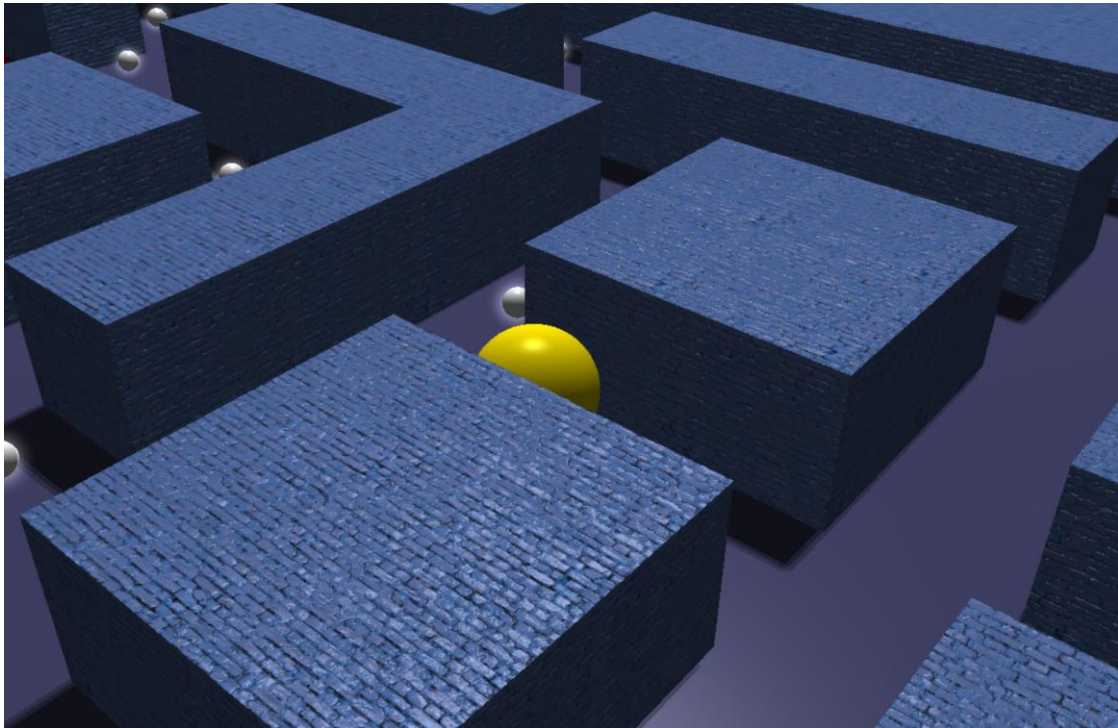


Cu shadow mapping

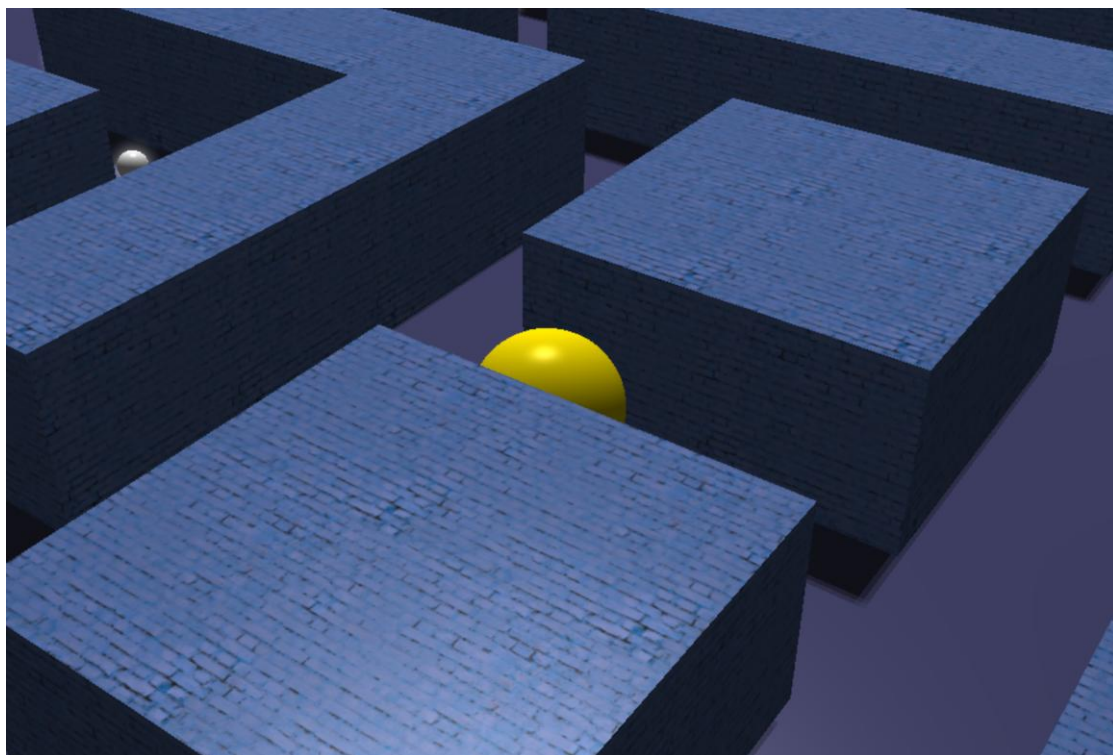


Fără shadow mapping

5. **Normal mapping în tangent space:** pe pereții texturați, cu lampă de cap pentru evidențierea detaliilor.

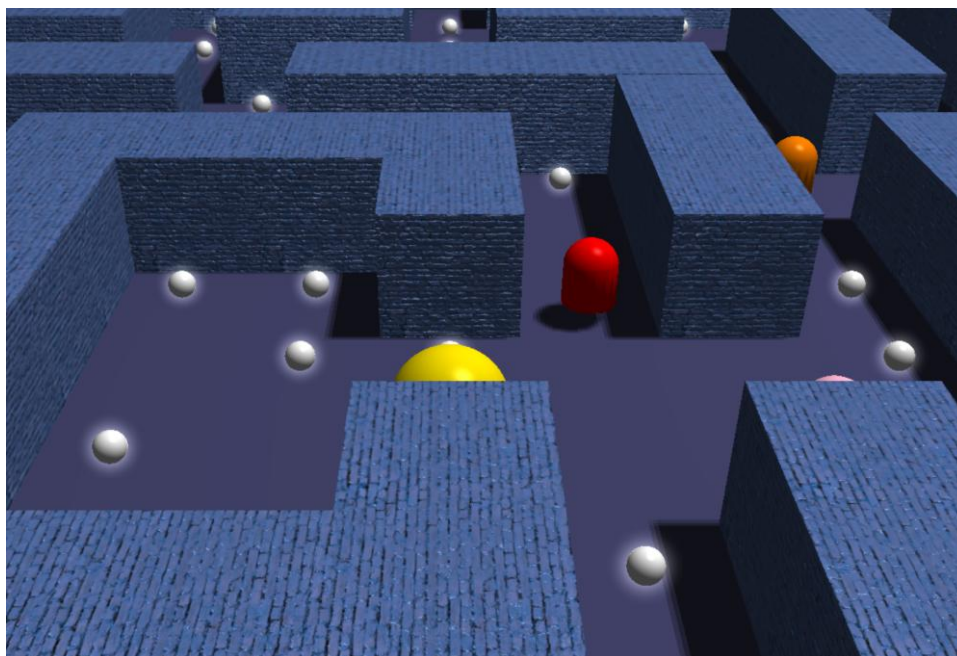


Cu normal mapping

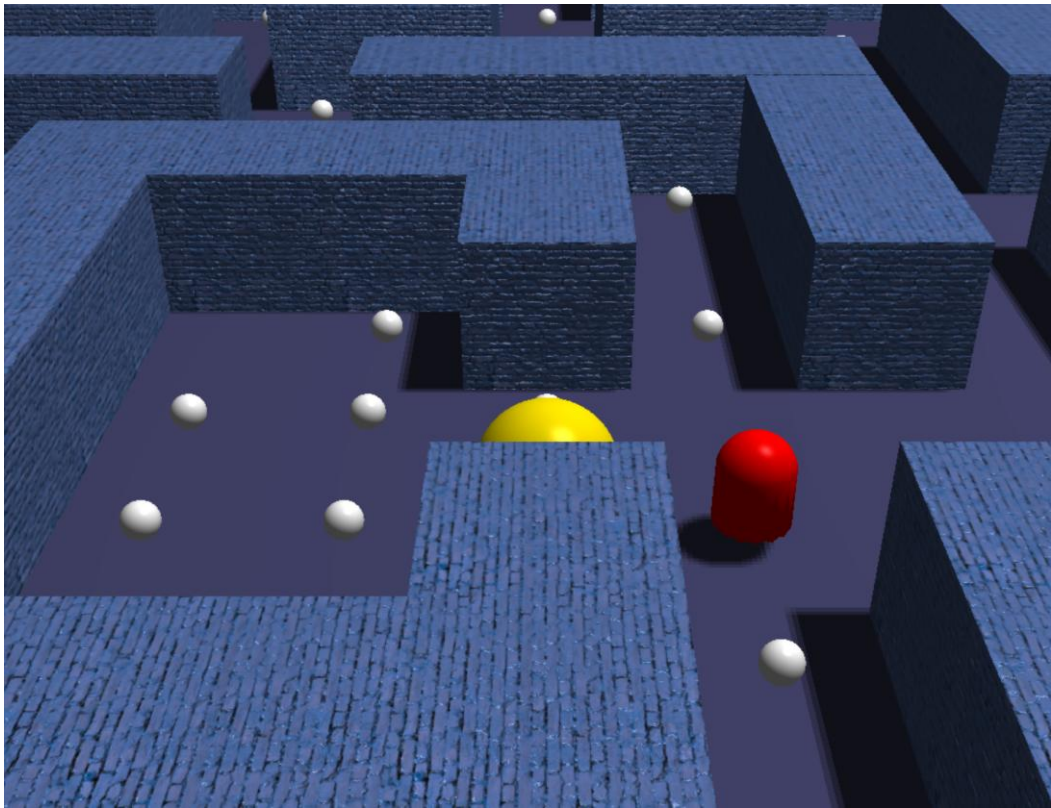


Fără normal mapping

6. **Halo aditiv pe bănuți:** prin billboard și blending aditiv.

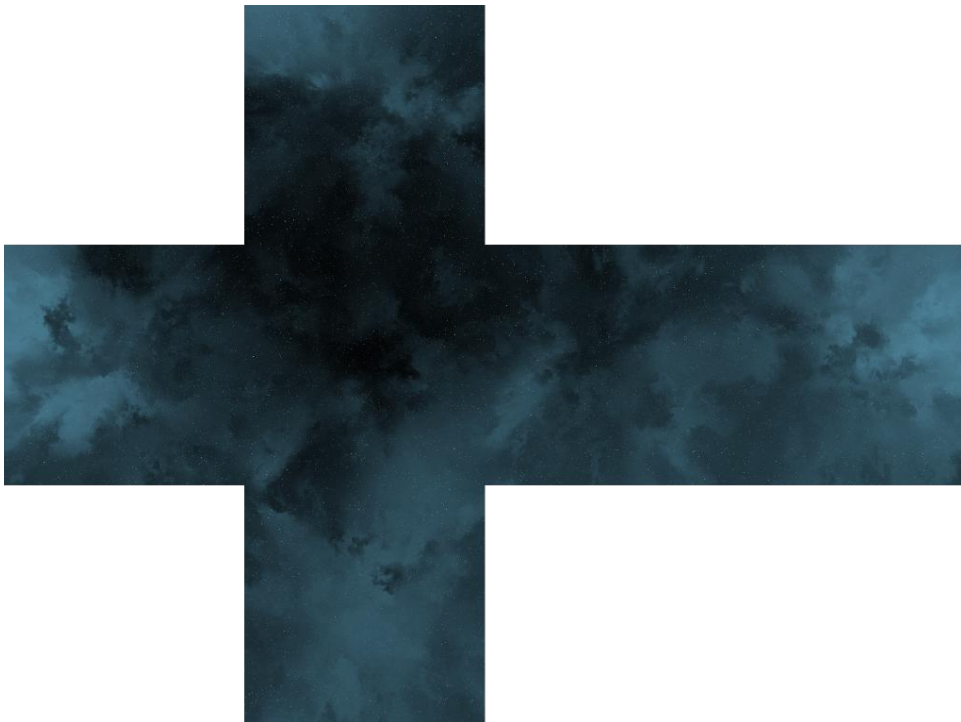


Cu glow

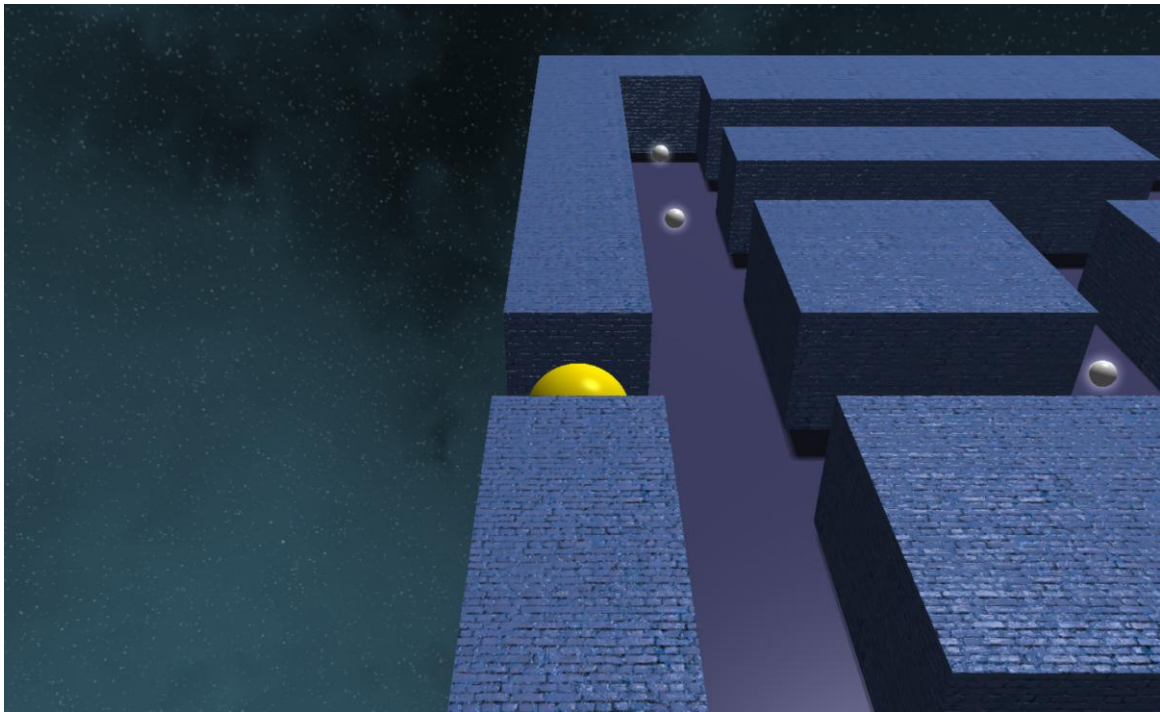


Fără glow

7. **Skybox cubemap:** randat în FBO + GL_LEQUAL pentru a trece testul de adâncime.



Cubemap-ul folosit



Marginea labirintului

8. **Camera care urmărește Pacman:** cu rotire prin click + drag mouse.
9. **Mișcare bazată pe deltaTime:** lerp pozițional și lerp unghiular.
10. **Toggle runtime:** N (Normal Mapping), G (Glow), M (Shadows), F (Fullscreen), R (Restart), ESC (Ieșire).

Comenzi si controale

- **W / Săgeata Sus:** Mută Pacman înainte
- **S / Săgeata Jos:** Mută Pacman înapoi
- **A / Săgeata Stânga:** Rotește Pacman cu 90 grade la stânga
- **D / Săgeata Dreapta:** Rotește Pacman cu 90 grade la dreapta
- **Click stâng + drag:** Rotește camera în jurul lui Pacman
- **N :** Activează / dezactivează Normal Mapping
- **G:** Activează / dezactivează Glow-ul bănușilor
- **M:** Activează / dezactivează Shadow Mapping
- **F:** Comuta modul Fullscreen
- **R:** Restartează jocul
- **ESC:** Închide aplicația

Concluzii

Proiectul a reprezentat o oportunitate de a integra și extinde lucrările de laborator într-un pipeline coerent și funcțional. Cele mai valoroase lecții tehnice au fost gestionarea corectă a shadow mapping-ului (bias adaptiv, filtrare PCF) și implementarea normal mapping-ului în tangent space, care a necesitat calculul al vectorilor tangenți și al matricei TBN în shader.

Dintre dificultățile întâmpinate, cele mai notabile au fost: alinierea coordonatelor de textură cu tangenții la generarea statică a geometriei prin greedy meshing, randarea skybox-ului fără a rescrie incorect depth buffer-ul și corecția lerp-ului unghiular pentru a evita rotații pe calea mai lungă de 180 de grade.

Contribuție personală

- **Skybox cu cubemap:** încărcarea celor 6 texturi PNG, configurarea `GL_TEXTURE_CUBE_MAP` și shaderele dedicate cu trick-ul `GL_LEQUAL`.
- **Sistemul de glow pentru bănuți:** soluția de billboard cu halo fading radial și blending aditiv, ca alternativă eficientă la bloom post-process.
- **Shadow mapping integral:** framebuffer offscreen, depth map 2048x2048, shaderul de adâncime separat și implementarea PCF în fragment shader.
- **Greedy meshing pentru pereți:** algoritmul de agregare a celulelor în segmente dreptunghiulare și generarea geometriei statice cu UV-uri scalate corect.
- **Toggle runtime pentru efecte grafice (N/G/M):** permite dezactivarea și activarea fiecărui efect, util pentru demonstrații comparative.
- **Modelul de iluminare phong și headlamp:** Implementarea componentelor ambientale, difuze și speculare, alături de o lumină frontală dinamică atașată camerei pentru a accentua detaliile peretilor texturați.
- **Normal mapping în spațiul tangent:** Calcularea matricii TBN în vertex shader și transpunerea vectorilor de lumină și vizualizare pentru un calcul corect al reliefului pe perete.
- **Logica de mișcare și interpolare fluidă:** Sistem de deplasare bazat pe lerping dependent de deltaTime pentru coordonatele și rotațiile lui Pacman, eliminând dependența de FPS.
- **Patrulare pentru fantome:** Logica de patrulare de tip round-robin pe rute, cu determinarea dinamică a unghiului de orientare la fiecare schimbare de direcție.
- **Gestiunea stării jocului și coliziuni:** Generarea pseudo-aleatoare a bănuților, calculul distanțelor pentru coliziuni cu labirintul / fantome, monitorizarea condiției de Win și logica de teleportare prin tunelul lateral.
- **Interfață 2D și control orbital:** Desenarea ecranelor de final prin dezactivarea temporară a testului de adâncime și randarea de text bitmap semi-transparent, plus sistemul de rotire a camerei în jurul jucătorului folosind mouse-ul.
- **Toggle runtime pentru efecte grafice (N/G/M) și cheat code:** Permite dezactivarea / activarea în timp real a normal mapping-ului, umbrelor și halo, adăugarea modului fullscreen, precum și a unei taste pentru victorie simplă.

Direcții de dezvoltare viitoare

- ✓ Comportament de urmărire a lui Pacman de către fantome (pathfinding BFS/A*).
- ✓ Sistem de vieți și scor vizibil în timp real.
- ✓ Bloom post-process și blur Gaussian.
- ✓ Animații procedurale: deschiderea gurii lui Pacman, animații pentru fantome.
- ✓ Generare procedurală a labirintului la fiecare joc nou.

Referințe Bibliografice

Toate laboratoarele de la disciplina „Sisteme de Prelucrare Grafică”

Texturi / Cubemap / Obiecte 3D

1. <https://tools.wwwtyro.net/space-3d/index.html#animationSpeed=1&fov=80&nebulae=true&pointStars=true&resolution=1024&seed=40pxpyc546i0&stars=true&sun=false>
2. <https://www.cgtrader.com/free-3d-models/character/other/pac-man-ghosts>
3. https://polyhaven.com/a/painted_brick

Tutoriale pe Learn OpenGL

1. <https://learnopengl.com/Getting-started/Coordinate-Systems>
2. <https://learnopengl.com/Lighting/Basic-Lighting>
3. <https://learnopengl.com/Getting-started/Textures>
4. <https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping>
5. <https://learnopengl.com/Advanced-OpenGL/Face-culling>
6. <https://learnopengl.com/Advanced-Lighting/Normal-Mapping>
7. <https://learnopengl.com/Advanced-OpenGL/Cubemaps>

Logica de lerp

1. https://www.reddit.com/r/Unity3D/comments/101ws3b/can_someone_explain_lerp/?solution=6476dbb8eaf45a8d6476dbb8eaf45a8d&js_challenge=1&token=bbbe4bf1c9a2b5160829c4be34da586133c1756409eb508763a0b8da92b6c6a8&jsc_orig_r=
2. <https://devforum.roblox.com/t/lerping-what-does-it-do-and-what-are-ways-i-can-use-it/1404206>
3. <https://devforum.roblox.com/t/consume-everything-how-greedy-meshing-works/452717>
4. <https://stackoverflow.com/questions/57645066/how-to-lerp-a-rotation-the-shortest-path>

Greedy Meshing / Pereții

1. https://www.reddit.com/r/VoxelGameDev/comments/cwtqtu/greedy_meshing_algorithm_implementation/
2. <https://gist.github.com/Vercidium/a3002bd083cce2bc854c9ff8f0118d33>
3. <https://kentpawson123.medium.com/procedural-generation-implementation-d9472d053cef>

DeltaTime

1. https://gafferongames.com/post/fix_your_timestep/
2. <https://discussions.unity.com/t/can-someone-explain-how-using-time-deltatime-as-t-in-a-lerp-actually-works/2554/2>

Billboard pentru bănuți / Texturi

1. <https://www.opengl-tutorial.org/intermediate-tutorials/billboards-particles/billboards/>
2. <https://gamedev.stackexchange.com/questions/113147/rotate-billboard-towards-camera>
3. <https://gamedev.stackexchange.com/questions/58737/odd-blending-result-semi-transparent-2d-quad-over-3d-scene>
4. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-5-a-textured-cube/>